

Modelo de Entidades e Relacionamentos (v1)

ESPECIFICAÇÃO DO PROJETO

ALUNO	Alex Oliveira · Disciplina: Computação Aplicada à Educação
VERSÃO	1.1 · SGBD: PostgreSQL 15 (Supabase)
DOCUMENTOS RELACIONADOS	Documentação do Projeto · Documento de Requisitos

1. Modelo conceitual

Oito entidades. Cinco são **catálogo** — carregadas pelo pipeline ou pelo autor do conteúdo, nunca alteradas pela aplicação. Três são **operacionais** — escritas durante o uso, sempre pelo servidor.

CÓDIGO	ENTIDADE	NATUREZA	ESCRITA POR
E-01	<code>usuarios</code>	operacional	servidor, no cadastro
E-02	<code>temas</code>	catálogo	autor do conteúdo
E-03	<code>categorias_defeito</code>	catálogo	pipeline
E-04	<code>programas_base</code>	catálogo	autor do conteúdo
E-05	<code>exercicios</code>	catálogo	pipeline
E-06	<code>dicas</code>	catálogo	autor do conteúdo
E-07	<code>tentativas</code>	operacional	servidor
E-08	<code>eventos</code>	operacional	servidor, somente inserção

1.1 Relacionamentos

RELACIONAMENTO	CARDINALIDADE	LEITURA
<code>temas</code> → <code>programas_base</code>	1 : N	um tema agrupa vários programas-base
<code>categorias_defeito</code> → <code>exercicios</code>	1 : N	uma categoria classifica vários exercícios
<code>categorias_defeito</code> → <code>dicas</code>	1 : 3	cada categoria tem exatamente três dicas
<code>programas_base</code> → <code>exercicios</code>	1 : N	de um programa saem vários exercícios, um por mutação
<code>usuarios</code> → <code>tentativas</code>	1 : N	um aluno faz várias tentativas
<code>exercicios</code> → <code>tentativas</code>	1 : N	um exercício é tentado várias vezes
<code>tentativas</code> → <code>eventos</code>	1 : N	uma tentativa acumula vários eventos

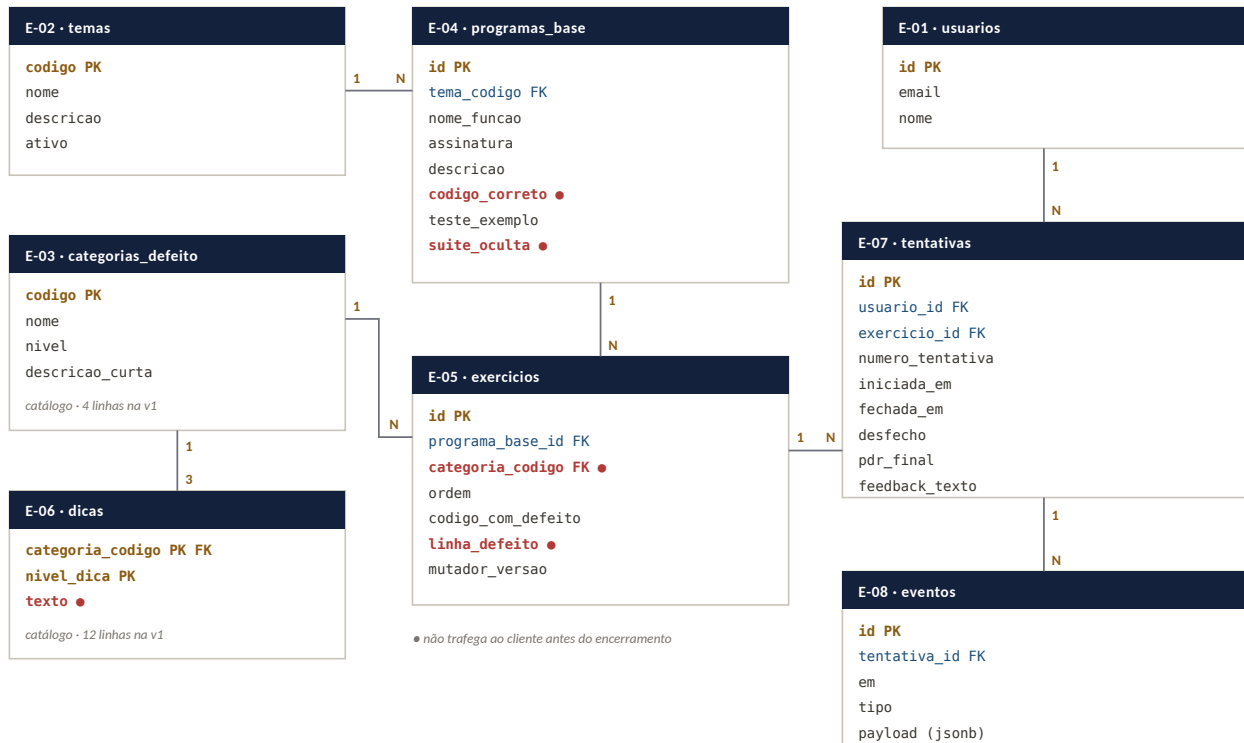
1.2 Duas decisões de modelagem

O nível de dificuldade pertence à categoria de defeito. Uma categoria tem dificuldade intrínseca: `comparador invertido` é sempre fácil de achar, `limite de laço deslocado` sempre exige simular o laço. Como o exercício herda a categoria do mutador que o produziu, colocar `nível` em `exercicios` duplicaria informação.

O tema pertence ao programa-base, não à categoria. `comparador invertido` ocorre igualmente em Fundamentos, POO e Estruturas de Dados — é o *programa* que pertence a um

tema, não o tipo de defeito. Esta é a razão de `programas_base` ser entidade própria em vez de colunas replicadas dentro de `exercicios`: além de carregar o tema, permite corrigir uma suíte de testes num lugar só e saber que dois exercícios vieram da mesma função.

2. Diagrama



3. Dicionário de dados

E-01 - usuarios

Espelha `auth.users`, gerenciada pelo Supabase Auth, acrescentando o nome de exibição. A senha nunca trafega nem é armazenada por esta tabela.

COLUNA	TIPO	NULO	DESCRIÇÃO
id	uuid PK	não	mesmo identificador de <code>auth.users.id</code>
email	text	não	único; usado para login
nome	text	não	nome de exibição; cabeçalho e feedback
criado_em	timestampz	não	padrão <code>now()</code> ; auditoria

E-02 • temas

Catálogo dos temas exibidos na tela de seleção, incluindo os que ainda não têm conteúdo. Atende RF-03.

COLUNA	TIPO	NULO	DESCRIÇÃO
codigo	text PK	não	ex. fundamentos
nome	text	não	ex. "Fundamentos de Programação"
descricao	text	não	uma linha, exibida no card
ativo	boolean	não	false marca o card como indisponível

Conteúdo da v1: fundamentos (ativo) · poo · ed1 · ed2 (inativos).

E-03 • categorias_defeito

Catálogo das categorias de erro que o pipeline sabe plantar. Define o nível e, por consequência, a pontuação-base.

COLUNA	TIPO	NULO	DESCRIÇÃO
codigo	text PK	não	identificador estável, ex. LACO_DESL
nome	text	não	ex. "limite de laço deslocado"
nivel	text	não	baixo ou medio ; restrito por CHECK
descricao_curta	text	não	uma frase; usada no feedback de reserva

Conteúdo da v1: CMP_INV (baixo) · ARIT_TROC (baixo) · LACO_DESL (medio) · ACUM_AUSENTE (medio).

E-04 • programas_base

Funções corretas das quais os exercícios derivam. Escritas à mão pelo autor do conteúdo.

COLUNA	TIPO	NULO	DESCRIÇÃO
id	uuid PK	não	
tema_codigo	text FK	não	referencia <code>temas.codigo</code>
nome_funcao	text	não	ex. <code>media_das_notas</code>
assinatura	text	não	ex. <code>media_das_notas(notas)</code> ; atende RF-06
descricao	text	não	enunciado: o que faz, entrada e saída
codigo_correto	text	não	forma canônica ; é a resposta do exercício
teste_exemplo	jsonb	não	<code>{chamada, entrada, esperado}</code> ; público
suite_oculta	text	não	código dos testes; nunca vai ao cliente

Sobre a forma canônica. `codigo_correto` armazena o resultado de `ast.unparse(ast.parse(fonte))`, não o texto como foi digitado. O pipeline compara o mutado contra essa forma para achar `linha defeito`; guardar o texto original faria a primeira divergência cair na linha 1, por causa da normalização de aspas, parênteses e comentários.

E-05 • **exercicios**

Gerada pelo pipeline. Cada linha é um programa-base com exatamente um nó da árvore alterado.

COLUNA	TIPO	NULO	DESCRIÇÃO
id	uuid PK	não	<code>uuid5</code> de chave natural; estável entre execuções
programa_base_id	uuid FK	não	referencia <code>programas_base.id</code>
categoria_codigo	text FK	não	referencia <code>categorias_defeito.codigo</code> ; é segredo
ordem	integer	não	sequência de apresentação (RN-09); único
codigo_com_defeito	text	não	o que o aluno vê no editor
linha_defeito	integer	não	1-indexado; verdade fundamental do diagnóstico
mutador-versao	text	não	versão do pipeline que gerou (RNF-08)
criado_em	timestampz	não	padrão <code>now()</code> ; auditoria

Por que `categoria_codigo` é segredo. A categoria é exatamente o conteúdo da dica de nível 1 — "a natureza do defeito". Se o cliente puder lê-la, a dica 1 deixa de custar pontos e a gradação de RN-04 vira enfeite.

E-06 • dicas

Textos fixos, um por combinação de categoria e nível. Chave primária composta.

COLUNA	TIPO	NULO	DESCRIÇÃO
categoria_codigo	text PK FK	não	referencia categorias_defeito.codigo
nivel_dica	smallint PK	não	1, 2 ou 3; restrito por CHECK
texto	text	não	exibido ao aluno; servido só pelo endpoint

E-07 • tentativas

Uma passagem do aluno por um exercício, do início ao encerramento.

COLUNA	TIPO	NULO	DESCRIÇÃO
id	uuid PK	não	
usuario_id	uuid FK	não	referencia usuarios.id
exercicio_id	uuid FK	não	referencia exercicios.id
numero_tentativa	smallint	não	atribuído pelo servidor conforme RN-11
iniciada_em	timestampz	não	padrão now()
fechada_em	timestampz	sim	nulo enquanto desfecho é aberto
desfecho	text	não	aberto, resolvido ou desistiu
pdr_final	integer	sim	cache derivado dos eventos; nulo enquanto aberta
feedback_texto	text	sim	gerado uma vez no encerramento (RF-15)

Sobre pdr_final. É cache, não fonte da verdade. Deve ser sempre reconstruível por `calcularPdr(eventos, {base, numero_tentativa})`. A coluna existe para que a consulta de total não reprocessse eventos de todos os alunos; sua ausência de autoridade é o que permite mudar os pesos e recalculer o histórico (RNF-09).

E-08 • eventos

Registro imutável, somente inserção, **gravado exclusivamente pelo servidor**. É a fonte da verdade do sistema.

COLUNA	TIPO	NULO	DESCRIÇÃO
<code>id</code>	<code>bigint</code> PK	não	identidade gerada; desempata eventos do mesmo instante
<code>tentativa_id</code>	<code>uuid</code> FK	não	referencia <code>tentativas.id</code>
<code>em</code>	<code>timestampz</code>	não	padrão <code>now()</code>
<code>tipo</code>	<code>text</code>	não	ver tabela abaixo
<code>payload</code>	<code>jsonb</code>	não	campos específicos do tipo

Tipos de evento e seus payloads

TIPO	PAYLOAD	ORIGEM
<code>localizou</code>	<code>{linha, correta, tentativa_num}</code>	RF-07
<code>editou</code>	<code>{codigo, linhas_alteradas}</code>	RF-08, RN-12
<code>precheck</code>	<code>{resultado, obtido, numero_uso}</code>	RF-09
<code>verificar</code>	<code>{resultado, passados, total}</code>	RF-10
<code>dica</code>	<code>{nivel}</code>	RF-12
<code>encerrou</code>	<code>{desfecho}</code>	RF-10, RF-13

Em `precheck` e `verificar`, `resultado` assume `passou`, `falhou`, `tempo_excedido` ou `erro`.

Por que `resultado` e não `passou: bool`. Um booleano não distingue "rodou e deu resultado errado" de "estourou o tempo" nem de "lançou exceção". A distinção importa para o feedback final e para a contagem de usos do Precheck.

Por que existe `encerrou` em vez de só `desistiu`. Com apenas `desistiu`, o desfecho `resolvido` teria de ser inferido de um `verificar` bem-sucedido — uma regra não escrita. Um único tipo de encerramento com o desfecho no payload torna a reconstrução do estado (RF-17) direta.

4. DDL

```
-- catálogo

create table temas (
  codigo      text primary key,
  nome        text not null,
  descricao   text not null,
  ativo       boolean not null default false
);

create table categorias_defeito (
  codigo      text primary key,
  nome        text not null,
  nivel       text not null check (nivel in ('baixo', 'medio')),
  descricao_curta text not null
);

create table programas_base (
  id          uuid primary key,
  tema_codigo text not null references temas(codigo),
  nome_funcao text not null,
  assinatura  text not null,
  descricao   text not null,
  codigo_correto text not null,
  teste_exemplo jsonb not null,
  suite_oculta text not null,
  unique (tema_codigo, nome_funcao)
);

create table exercicios (
  id          uuid primary key,
  programa_base_id uuid not null references programas_base(id),
  categoria_codigo text not null references categorias_defeito(codigo),
  ordem       integer not null unique,
  codigo_com_defeito text not null,
  linha_defeito integer not null check (linha_defeito > 0),
  mutador_versao text not null,
  criado_em   timestampz not null default now()
);

create table dicas (
  categoria_codigo text not null references categorias_defeito(codigo),
  nivel_dica       smallint not null check (nivel_dica between 1 and 3),
  texto           text not null,
  primary key (categoria_codigo, nivel_dica)
);

-- operacional

create table usuarios (
  id          uuid primary key references auth.users(id) on delete cascade,
  email       text not null unique,
  nome        text not null,
  criado_em   timestampz not null default now()
);

create table tentativas (
  id          uuid primary key default gen_random_uuid(),
  usuario_id  uuid not null references usuarios(id) on delete restrict,
  exercicio_id uuid not null references exercicios(id) on delete restrict,
  numero_tentativa smallint not null check (numero_tentativa > 0),
  iniciada_em timestampz not null default now(),
  fechada_em  timestampz,
  desfecho    text not null default 'aberto'
  check (desfecho in ('aberto', 'resolvido', 'desistiu')),
  pdr_final   integer check (pdr_final >= 0),
  feedback_texto text,

  constraint coerencia_fechamento check (
    (desfecho = 'aberto' and fechada_em is null and pdr_final is null) or
    (desfecho <> 'aberto' and fechada_em is not null and pdr_final is not null)
  ),
  constraint tentativa_unica unique (usuario_id, exercicio_id, numero_tentativa)
```

```

);

create table eventos (
  id          bigint primary key generated always as identity,
  tentativa_id  uuid not null references tentativas(id) on delete restrict,
  em          timestampz not null default now(),
  tipo        text not null check (tipo in
    ('localizou','editou','precheck','verificar','dica','encerrou')),
  payload      jsonb not null default '{} '::jsonb
);

-- ----- índices -----

create index idx_programas_tema      on programas_base (tema_codigo);
create index idx_exercicios_programa on exercicios (programa_base_id);
create index idx_exercicios_categoria on exercicios (categoria_codigo);
create index idx_tentativas_usuario  on tentativas (usuario_id);
create index idx_tentativas_exercicio on tentativas (exercicio_id);
create index idx_tentativas_fechadas  on tentativas (usuario_id, fechada_em)
  where desfecho <> 'aberto';
create index idx_eventos_tentativa    on eventos (tentativa_id, em, id);

-- só uma tentativa aberta por aluno e exercício (RN-10)
create unique index idx_uma_aberta_por_exercicio
  on tentativas (usuario_id, exercicio_id)
  where desfecho = 'aberto';

```

`exercicios.ordem` já ganha índice por ser `unique`; não há índice adicional sobre ela.

As chaves estrangeiras de `tentativas` e `eventos` usam `on delete restrict`, não `cascade`. Apagar um usuário não pode remover silenciosamente o histórico que DA-06 define como fonte da verdade — excluir um aluno passa a exigir decisão deliberada sobre o que fazer com as tentativas dele.

4.1 Integridade que o SQL declarativo não cobre

Exatamente três dicas por categoria. Verificado pelo pipeline e por este teste, que deve rodar após cada carga:

```

select c.codigo, count(d.nivel_dica) as qtd
from categorias defeito c
left join dicas d on d.categoria_codigo = c.codigo
group by c.codigo
having count(d.nivel_dica) <> 3;
-- deve retornar zero linhas

```

linha defeito dentro do código. Exige contar linhas de outra coluna, o que um CHECK não faz. Verificado pelo pipeline na geração.

5. Segurança de acesso

Atende RNF-03, RNF-04 e a decisão DA-07. O princípio: **o cliente lê pouco e não escreve nada.**

5.1 O que o cliente pode ler

Nenhuma tabela é exposta diretamente. A leitura passa por duas views; o resto chega por endpoint.

```
create view temas_publicos as
  select codigo, nome, descricao, ativo from temas;

-- omite categoria_codigo (= dica 1), linha defeito,
-- codigo_correto e suite_oculta
create view exercicios_publicos as
  select e.id, e.ordem, e.codigo_com_defeito,
         p.assinatura, p.descricao, p.teste_exemplo,
         p.tema_codigo, c.nivel
  from exercicios e
  join programas_base p      on p.id = e.programa_base_id
  join categorias_defeito c on c.codigo = e.categoria_codigo;
```

`nivel` é exposto porque o aluno escolheu o nível e precisa ver a pontuação-base. `categoria_codigo` não, porque revelaria a natureza do defeito.

5.2 Permissões

```
-- o papel do cliente não enxerga tabela nenhuma
revoke all on temas, categorias_defeito, programas_base,
          exercicios, dicas, tentativas, eventos
  from anon, authenticated;

-- só as views, e só leitura
grant select on temas_publicos, exercicios_publicos to authenticated;

-- o perfil próprio, para o cabeçalho
grant select, update on usuarios to authenticated;
```

`dicas` **não tem grant algum para o cliente**. O texto chega exclusivamente por `POST /api/dica`, que confere a condição de liberação de RN-04 antes de responder. Sem isso, o aluno leria os três textos de uma vez e o custo em pontos seria decorativo.

`tentativas` e `eventos` **também não recebem grant de escrita**. Tudo passa pelos endpoints com a chave de serviço. Se o cliente pudesse escrever, gravaria `desfecho = 'resolvido'` com `pdr_final` arbitrário, ou um `localizou` com `correta: true` — e a pontuação perderia sentido.

5.3 Row Level Security

O RLS fica ligado como segunda barreira, para o caso de um grant ser concedido por engano no futuro.

```

alter table usuarios    enable row level security;
alter table tentativas enable row level security;
alter table eventos    enable row level security;

create policy usuario_proprio on usuarios
  for all to authenticated
  using (id = auth.uid()) with check (id = auth.uid());

create policy tentativas_leitura on tentativas
  for select to authenticated using (usuario_id = auth.uid());

create policy eventos_leitura on eventos
  for select to authenticated using (
    exists (select 1 from tentativas t
            where t.id = eventos.tentativa_id and t.usuario_id = auth.uid())
  );

```

Não existe política de `insert`, `update` ou `delete` para `authenticated` sobre `tentativas` e `eventos`. Somado à ausência de `grant`, a escrita pelo cliente fica impossível por dois mecanismos independentes.

A chave de serviço usada pelo servidor ignora RLS por definição. É por isso que ela nunca pode chegar ao navegador nem ao processo que executa código de aluno (RF-11).

5.4 O gabarito após o encerramento

A tela de feedback precisa de `codigo_correto` e `linha defeito`, que as views escondem. São servidos por `GET /api/gabarito/[tentativa_id]`, que só responde quando a tentativa pertence ao solicitante e tem `desfecho <> 'aberto'`.

6. Consultas de referência

```
-- PDR acumulado do aluno
select coalesce(sum(pdr_final), 0) as pdr_total
from tentativas
where usuario_id = $1 and desfecho <> 'aberto';

-- progresso no tema (RF-03)
select count(*) as total,
       count(r.ok) as resolvidos
from exercicios e
join programas_base p on p.id = e.programa_base_id
left join lateral (
  select 1 as ok from tentativas t
  where t.exercicio_id = e.id and t.usuario_id = $1
        and t.desfecho = 'resolvido' limit 1
) r on true
where p.tema_codigo = $2;

-- RN-09, passo 1: existe tentativa aberta no tema e nível?
select t.id, t.exercicio_id
from tentativas t
join exercicios e      on e.id = t.exercicio_id
join categorias defeito c on c.codigo = e.categoria_codigo
join programas_base p   on p.id = e.programa_base_id
where t.usuario_id = $1 and t.desfecho = 'aberto'
      and p.tema_codigo = $2 and c.nivel = $3
limit 1;

-- RN-09, passo 2: não resolvido de menor ordem;
-- esgotados, o de menor ordem entre os resolvidos
select e.id, (r.ok is not null) as ja_resolvido
from exercicios e
join categorias defeito c on c.codigo = e.categoria_codigo
join programas_base p   on p.id = e.programa_base_id
left join lateral (
  select 1 as ok from tentativas t
  where t.exercicio_id = e.id and t.usuario_id = $1
        and t.desfecho = 'resolvido' limit 1
) r on true
where p.tema_codigo = $2 and c.nivel = $3
order by (r.ok is not null), e.ordem
limit 1;

-- eventos de uma tentativa, para recalculer o PDR
select tipo, payload, em
from eventos
where tentativa_id = $1
order by em, id;

-- próximo numero_tentativa (RN-11), na transação que cria a tentativa
select coalesce(max(numero_tentativa), 0) + 1
from tentativas
where usuario_id = $1 and exercicio_id = $2;

-- desempenho por categoria, base do perfil quando ele entrar (fora da v1)
select c.nome,
       count(*) filter (where t.desfecho = 'resolvido') as resolvidos,
       round(avg(t.pdr_final)) as pdr_medio
from tentativas t
join exercicios e      on e.id = t.exercicio_id
join categorias defeito c on c.codigo = e.categoria_codigo
where t.usuario_id = $1 and t.desfecho <> 'aberto'
group by c.nome
order by pdr_medio;
```

O `order by (r.ok is not null), e.ordem` do passo 2 implementa RN-09 por inteiro: `false` vem antes de `true`, então não resolvidos primeiro, e dentro de cada grupo pela ordem. Quando tudo está resolvido, devolve o de menor ordem em vez de nada — que é o comportamento exigido pelo segundo critério de aceitação de RF-05.

7. Notas de evolução

ITEM FUTURO	O QUE JÁ ESTÁ PRONTO	O QUE FALTA
Elo e janela móvel	<code>pdr_final</code> por tentativa, ordenável por <code>fechada_em</code>	consulta de média sobre as 20 últimas
Ranking semanal	<code>fechada_em</code> indexado	agregação com recorte de data
Ranking geral	idem	depende do elo
Perfil por categoria	consulta da seção 6 pronta	tela
Nível Alto	<code>nivel</code> é CHECK com dois valores	acrescentar <code>alto</code> ao CHECK e as categorias
Novos temas	<code>temas</code> e <code>programas_base.tema_codigo</code> existem	escrever os programas-base e marcar <code>ativo</code>
Dicas sob medida	<code>dicas</code> como tabela, servida por endpoint	geração dinâmica coexistindo com o texto fixo

Nenhum item exige mudança estrutural — todos são inserção de linhas, uma consulta nova ou uma tela.